CPSC 436Q Project

Matthew Jacobs, Michael Leung
University of British Columbia

Abstract

We will be exploring the Density Matrix Renormalization Group (DMRG) algorithm and implementing it on a 1D spin-$\frac{1}{2}$ chain under the Heisenberg model. We will first go through the physical motivations behind the DMRG algorithm and define the Heisenberg model. We will then describe the DMRG algorithm in detail. Next, we will go through our own implementation of DMRG on a 1D spin-$\frac{1}{2}$ chain under the Heisenberg model using Python. Furthermore, we will include various graphs using results from our algorithm and interpret them. Finally, we will discuss the limitations of both our implementation and DMRG in general (for example, in the case of a 2D simulation).

1 Introduction

Simulating many-body quantum systems is a challenging problem in physics. Even simple models, such as chains of interacting spins, quickly become computationally infeasible due to the exponential growth of configurations [DCRRM08]. To overcome this, approximation techniques have become essential. One of the most powerful of these techniques is the Density Matrix Renormalization Group (DMRG) algorithm.

In Section 2, we introduce the physical motivations behind the DMRG algorithm, and in Section 3, we introduce the Heisenberg model. We describe the DMRG algorithm in detail in Section 4. Section 5 outlines our own implementation of the infinite-system DMRG algorithm on a 1D spin-$\frac{1}{2}$ chain under the Heisenberg model using Python, and we interpret various results from our algorithm graphically in Section 6. Finally, in Section 7, we discuss the limitations of both our implementation and DMRG in general.

2 Physical Motivations

In condensed matter physics, many systems can be described as quantum particles arranged on a lattice [DCRRM08]. However, when multiple particles interact, the number of possible configurations grows exponentially. For example, a line of n spin-$\frac{1}{2}$ particles has 2^n possible configurations [DCRRM08]. This means that directly solving problems is limited to relatively small systems.

Therefore, to study the low energy properties (the behaviours at the ground state or other low energy states) of these systems, one can use the Density Matrix Renormalization Group (DMRG) algorithm, introduced by Steven R. White in 1992 [Whi92]. It was originally designed for 1D systems, but can also be adapted to simulate small 2D systems [DCRRM08].

DMRG builds an approximate representation of the system by keeping only the most important basis states [Whi92]. Physically, this means DMRG focuses on the low energy, strongly correlated configurations of the system, while discarding high energy or weakly contributing states. This keeps the algorithm efficient.

3 The Heisenberg Model

The Heisenberg Model is a statistical model that describes the interactions between spins in a lattice for a magnetic system. For example, it can be used to study phase transitions for magnetic systems [Pir21]. It is represented by the Hamiltonian:

$$H = J \sum_{\langle i,j \rangle} \overrightarrow{S_i} \overrightarrow{S_j}$$

Where $\langle i, j \rangle$ represents the unordered pairs of i and j. This means it runs over each pair of spins in the lattice. $\overrightarrow{S_i}$ represents the spins of site i. Finally, J represents the exchange constants, which describe the properties of the model [Pir21].

This Hamiltonian can be expanded to:

$$H = \sum_{\langle i,j \rangle} J^x S_i^x S_j^x + J^y S_i^y S_j^y + J^z S_i^z S_j^z$$

Thus, depending on the constants J, the following models are defined [Pir21]:

- Heisenberg XYZ Model for $J^x \neq J^y \neq J^z$
- Heisenberg XXZ Model for $J^x = J^y \neq J^z$
- Heisenberg XXX Model for $J^x = J^y = J^z$

We will be using these models to compare and contrast results obtained by our implementation.

4 The DMRG Algorithm

4.1 History

The Density Matrix Renormalization Group (DMRG) algorithm in its simplest form as first conceived by White [Whi92] is a modification of the Renormalization Group techniques used by Wilson to solve the Kondo problem [Wil75]. The central idea involves breaking down the analysis of a system into a sequence of intermediate steps operating on a subset of the system, and truncating the representation at that point in order for the solution to remain computationally feasible over many iterations. Typically it is used to find the ground state of a system. The key difference between approaches is the choice of which basis to truncate to.

Wilson's original method, which is sometimes referred to as truncated iterative diagonalization (TID), chooses the new basis to be that which is spanned by the m lowest-energy eigenstates of the current subsystem, where m is the dimension of the original small subsystem's Hamiltonian [CM23]. This proved to be successful for quantum impurity problems, but little else, because it assumes that the ground state of the system as a whole is composed of the low-energy states of subsystems contained within [DCRRM08].

In order to generalize this approach, White proposed using the basis described by the m eigenvectors with the largest eigenvalues in the density matrix of the subsystem [Whi92]. The specific procedure is described in full in the following subsection.

4.2 The Infinite-System Algorithm

There are essentially two versions of the DMRG algorithm, for analyzing either infinite or finite systems. We start by describing the infinite-system algorithm, as it is the simplest, and the finite-system algorithm builds upon it.

Of course, we are unable to directly model an infinite number of particles, so we begin by choosing a fixed value of n particles which will approximate an infinite chain. The initial state of the algorithm consists of two "blocks" representing some subsystem, often simply a single particle or "site." We refer to these as the system block S (not to be confused with the overall system under observation) and environment block E, but the specific label assigned to each block is arbitrary. Each of these blocks is represented by a Hamiltonian, its basis size, and a set of operators which describe its interactions with other sites. We then increase the system size by two sites, adding one each to S and E to create the enlarged left and right blocks, L and R. This is done by creating a new Hamiltonian which consists of the block's Hamiltonian, the site's, and one describing the interactions between the block and the site. Mathematically, if we consider forming L by combining S with a site labelled a, we have:

$$\hat{H}_L = \hat{H}_S + \hat{H}_a + \hat{H}_{Sa}. \tag{1}$$

We then follow a similar procedure, merging L and R to create the "superblock," where the only interaction occurs between the two single sites, a and b, which were just added:

$$\hat{H}_{super} = \hat{H}_L + \hat{H}_R + \hat{H}_{ab}. \tag{2}$$

At this point, if the starting blocks S and E at iteration i had a basis size of m_i, and single particles exist in a basis of size d, then the superblock describing all of the components at iteration i exists in an $m_i^2 \times d^2$ basis. Clearly, the number of iterations is directly and linearly correlated to the number of particles under observation, so without truncation, the analysis would scale exponentially in terms of the number of particles n, $O(d^n)$.

In order to truncate the numerical representation, we first diagonalize $\hat{H}_{super}$ and choose the eigenstate $|\psi\rangle$ which represents the energy state that is the target of our analysis. Typically, $|\psi\rangle$ is chosen to be the ground state, but DMRG also works to target other low-energy eigenstates. Using this target state, we create the density matrix $\rho = |\psi\rangle \langle\psi|$, reduce it to the subspace of L (and R), and diagonalize to find the eigenstates. From here, we choose the m eigenstates with the highest eigenvalues (where m is fixed *a priori*) to represent the reduced basis.

These eigenstates are used as the columns to form a rectangular transformation matrix O_L (and O_R), which is used to truncate the left (and right) blocks as follows:

$$\hat{H}'_L = O^\dagger \hat{H}_L O. \tag{3}$$

These then serve as the system and environment blocks which are given as input to the next iteration ($\hat{H}'_S = \hat{H}'_L$). We continue iterating until the total number of particles represented across S and E is equal to our hyperparameter n.

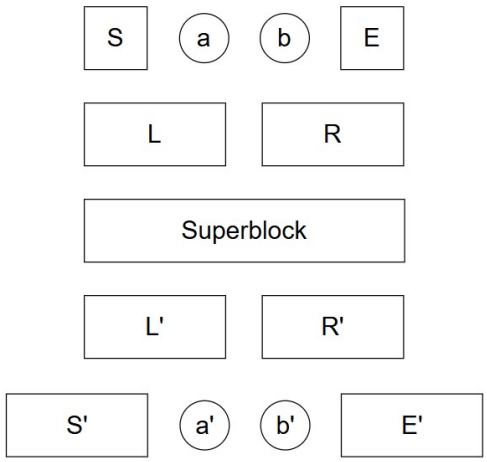

Figure 1: A schematic representation of a single step of the infinite-system DMRG algorithm.

4.3 The Finite-System Algorithm

If the goal is to analyze a system of finite size, it may seem as simple as setting n to be the desired number of particles. However, this is not very accurate, because the initial density matrices used are derived from small lattices (we don't yet have n particles), and so the targeted wave function is different at each step and may not in fact be a good approximation for the entire system [Whi92] [CM23] [Sch11].

The first step of the finite algorithm is indeed to execute the infinite algorithm up to n sites. The result is then refined by performing some number of "sweeps" (a hyperparameter), which are described by the following procedure. Keeping a fixed system length of n, split the system at the middle into the same step configuration as above: a system block S, two free sites a and b, and an environment block E. In the growth phase (running the infinite algorithm), we would be adding new free sites to the middle, but here we are breaking apart the system without introducing anything new. Specifically, we extend the system by adding a to S as before, creating the enlarged left block L, and truncate it to make the new system block. However, the right block R is shrunk instead, simply by removing b from it (this can be accomplished by storing and retrieving the smaller block that was already computed during the growth phase), creating the new environment block. This process is repeated until the environment is shrunk to a single site, then reversed, where E is allowed to grow and S shrinks. Again, this terminates when S is a single site. Finally, allow S to grow again until it reaches the middle of the system; this constitutes one sweep. Repeat for the desired number of sweeps, decided *a priori*.

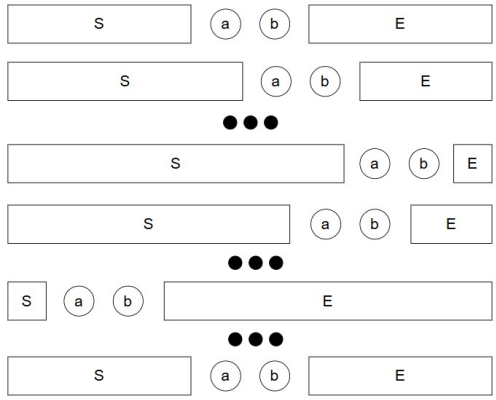

Figure 2: A schematic representation of the sweep procedure for the finite-system DMRG algorithm.

5 Our Implementation

The code for our implementation can be found here. To run, simply install Python, download the repository, and run `python main.py` from the root directory of the repository. This will use our DMRG implementation to analyze various small systems, and generate the graphs presented in Section 6 (under the "visual" folder).

We implemented the simplest version of the algorithm, to make sure we truly understood it in its original formulation. [GM17] was a valuable resource for this. However, modern techniques have been devised which can improve the utilization of DMRG, some of which are described in [Sch11] and demonstrated in [HUA+24].

The actual algorithmic logic lives in `main.py`, but first a brief word on two other files holding important structural information. The file `dmrg_classes.py` provides a convenient way to store all the relevant information of a subsystem at any step of the iteration, most notably the Hamiltonian describing it. On the other hand, `dmrg_operators.py` stores the basic operators that describe a single site as constants which can be accessed from `main.py`, as well as a function which computes the Hamiltonian describing the interactions between two given blocks.

Within `main.py`, the second half of the file is concentrated on producing various visuals, so only the first half is relevant to analyzing the actual implementation of the algorithm. The function `infinite_dmrg` serves to kickoff the algorithm with the provided parameters:

- `sites`: the desired number of sites to be included in the final system (which should approximate an infinitely long system),
- `keep`: the maximum dimension allowed for Hamiltonians; the number of eigenstates to keep in the reduced basis,
- `start`: a `Block` object representing the starting subsystem (both S and E), and
- `Jx, Jy, Jz`: the coupling constants which are applied to the spin terms when constructing the Hamiltonian.

The main loop of `infinite_dmrg` simply serves to iterate through the steps until the desired system size is reached. This is done by calling the `step` function, and updating the block accordingly. Inside the `step` function is where most of the logic takes place. It first calls helper functions to enlarge `system` and `env` by a single site, then merge them together. Then, `scipy`'s eigensolver `eigsh` is used to find the single eigenstate with the smallest algebraic eigenvalue, which is the ground state. The returned wavefunction is then reshaped into a matrix A, and we "trace out" the parts relating to R, so that the ket-bra product $\rho = AA^\dagger$ represents the density matrix of only the left block (L).

This density matrix is then passed to a helper function `diagonalize_matrix`, which calls another library-provided eigensolver and returns the eigenstates as pairs of eigenvalues and eigenvectors. These are sorted by their eigenvalues in decreasing order so that we can easily extract the m most important eigenstates by indexing the array up to m. At this point, we are ready to create the transformation matrix O. We start by creating a zero-matrix of the appropriate shape, then iteratively replace each column with the selected eigenvectors. Finally, we use the transformation matrix to truncate the matrices of L, and create the new block which is the result of this step. We also return the energy of the ground state, which was calculated from the superblock earlier.

6 Results and Visualization

We now analyze the visualizations generated by our program. We use energy per site to make results comparable between different system sizes. It is calculated by the formula $E_{site} = E/L$, where E is the total energy, and L is the number of sites.

6.1 Energy Per Site vs Number of Sites

The first visual is plotting the energy per site versus the number of sites for each of the different Heisenberg Models. Since this is an implementation of the infinite-system DMRG algorithm, we suspect that increasing the number of sites will cause the results to converge to the true energy value. This is indeed what we observe in Figure 3.

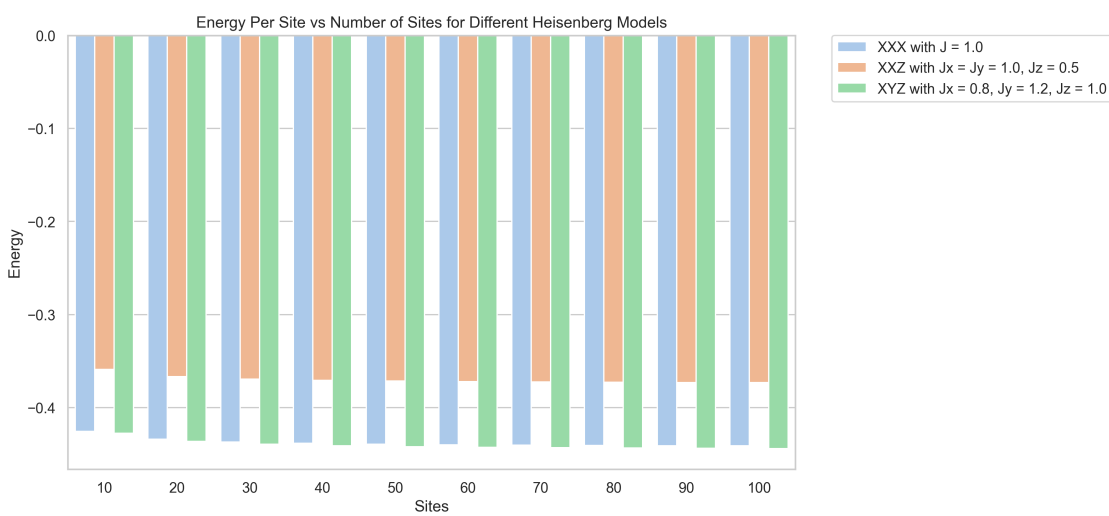

Figure 3: Energy per site as a function of how many sites are used to model an infinite system, for three different Heisenberg models.

While the difference is small, Figure 4 gives a clear convergence trend.

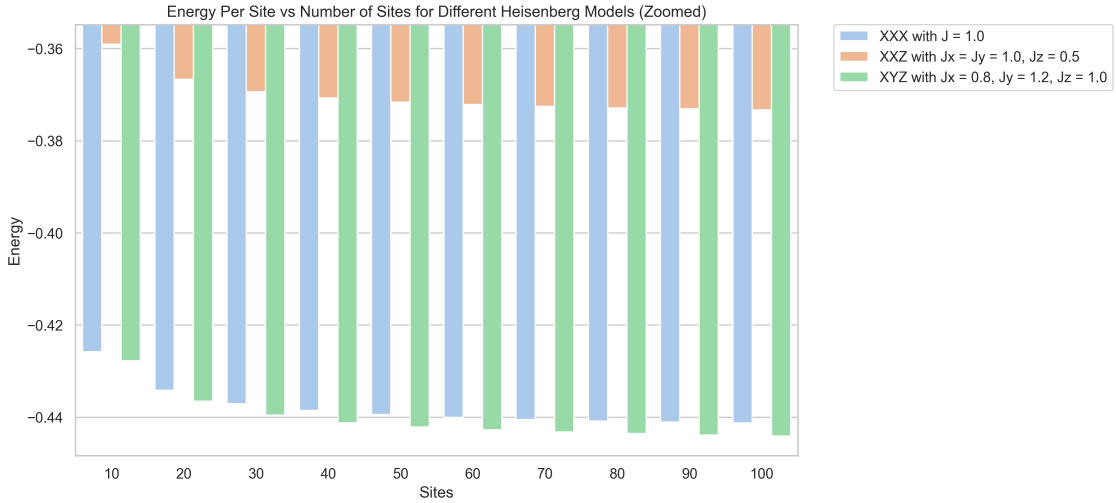

Figure 4: A zoomed-in version of the graph from Figure 3.

The value converges as the number of sites increases, because in each iteration of DMRG, we enlarge the system and recompute an effective basis that captures the correlations between neighboring blocks. Allowing more sites means that boundary effects become less significant and the bulk region of the chain dominates, producing a more accurate value.

6.2 Energy Per Site vs J in the XXX Model

The second visual (Fig. 5) is plotting the energy per site versus J in the XXX model. We choose the range of J as -2 to 2 to show the difference between a ferromagnetic system to an antiferromagnetic system. As a reminder, the XXX model is when $J^x = J^y = J^z = J$.

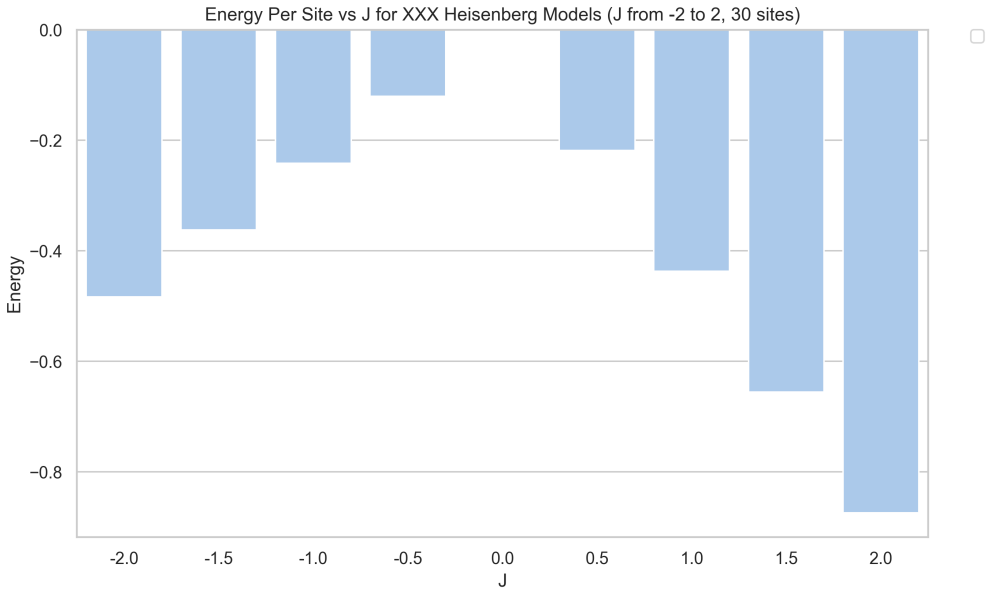

Figure 5: Energy per site for different values of the exchange constant J.

Something that draws attention is the asymmetry. At $J = -2.0$, the energy per site is higher than at $J = 2.0$, and the negative values have a more gradual change in energy. Physically, when J is smaller than 0, energy is minimized when the spins are parallel to each other (ferromagnet). When J is greater than 0, the energy is minimized when they are antiparallel (antiferromagnet)[Pir21]. Quantum effects lower the energy of an antiferromagnet system, but not a ferromagnet system [vV16]. We can see this reflected in the graph.

6.3 Energy Per Site vs J^x and J^z in the XXZ Model

The third visual (Fig. 6) is a heatmap of energy per site versus J^x and J^z in the XXZ model. We choose the range of J^x and J^z as -2 to 2. As a reminder, the XXZ model is when $J^x = J^y \neq J^z$.

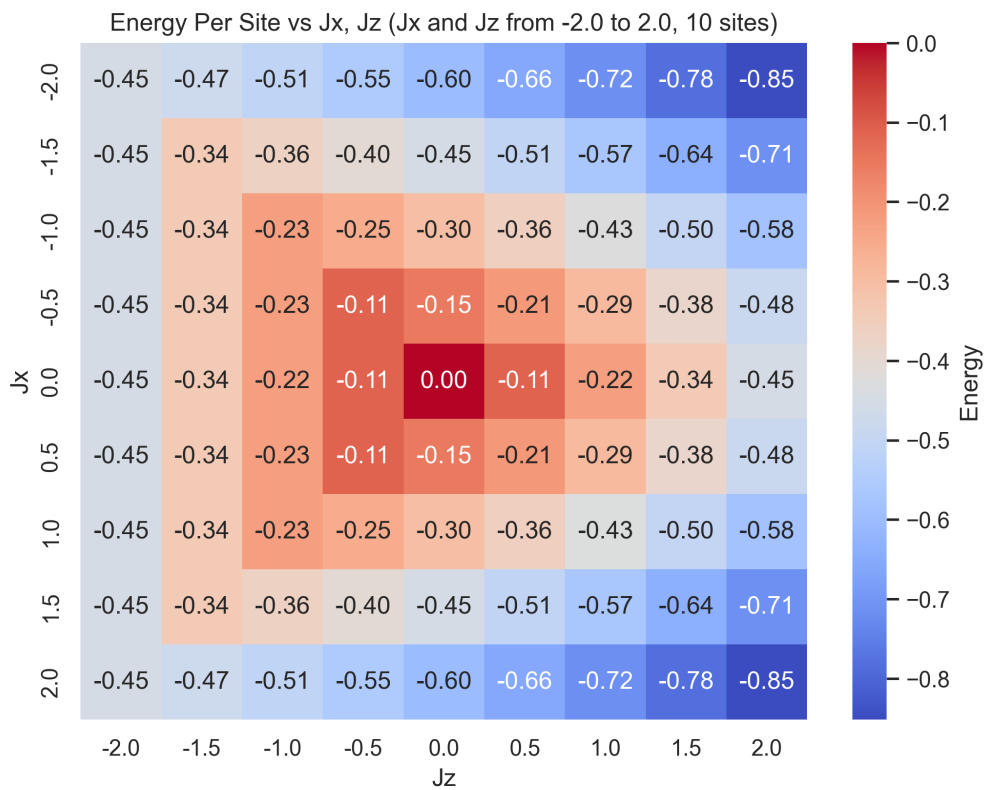

Figure 6: Energy per site for different values of the exchange constants J^x and J^z.

From a column viewpoint, we can see that deviating from $J^x = 0$ decreases the energy per site symmetrically. This is because the XXZ Hamiltonian is even in J^x:

$H = \sum_{\langle i,j \rangle} J^z S_i^z S_j^z$

This is the same as an Ising model without an external field, which is symmetric. However, at all other points of J^z it follows a similar pattern seen in Section 6.2. The energy increases to a certain inflection point, then starts decreasing asymmetrically (the left has higher energy per site than the right side). This occurs because a negative J^z competes with the J^x terms, while a positive J^z cooperates with them. As J^x deviates from 0, the inflection point shifts leftward due to stronger antiferromagnetic correlations.

6.4 Energy Per Site vs Basis Dimension

The fourth visual (Fig. 7) compares energy per site to the dimension of the reduced basis. Equivalently, this is the number of eigenstates kept after each truncation. Thus, we expect that, as we increase the size of the basis, the energy per site value will converge. Keep in mind that this graph is zoomed in.

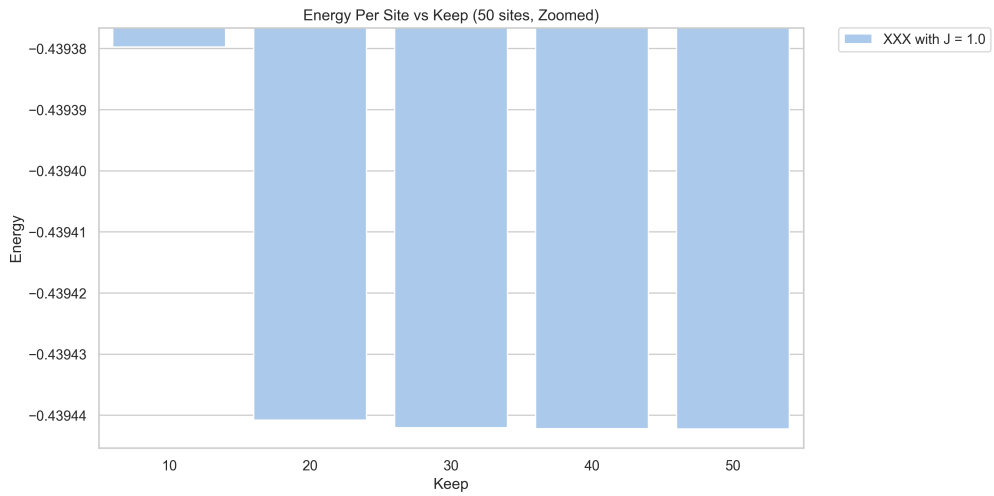

Figure 7: Energy per site as a function of the dimension of the reduced basis.

The above analysis uses $n = 50$ sites, because if the number of sites is too small, even keeping only 10 eigenstates is sufficient to capture the full system. For 50 sites, however, truncating more eigenstates has a noticeable effect: the energy becomes less accurate as information is discarded. In this case, we can see that at around 30 eigenstates, the full system information is captured.

7 Conclusion

The Density Matrix Renormalization Group (DMRG) remains one of the most powerful numerical methods for studying 1D quantum systems. As seen in our repository, it is simple to implement the infinite-system DMRG algorithm in Python. However, DMRG is not without weaknesses. For example, in 2D or 3D systems, entanglement grows much faster, and the number of states needed to maintain accuracy grows exponentially. Thus, DMRG is best suited for 1D operations [DCRRM08]. Due to this limitation, a variety of alternatives and extensions of DMRG have been proposed. For example, MERA builds on the DMRG algorithm by incorporating "disentanglers" to handle the larger entanglement present in higher dimensional systems [Vid08]. Overall, DMRG is best viewed as an efficient and accurate tool for studying 1D quantum systems, whose strengths and limitations are well understood. Through our graphical analysis, one can see how the infinite-system DMRG algorithm can be correctly and straightforwardly implemented in Python.

References

[CM23] G. Catarina and Bruno Murta. Density-matrix renormalization group: a pedagogical introduction. *The European Physical Journal B*, 96(8), August 2023. URL: http://dx.doi.org/10.1140/epjb/s10051-023-00575-2, doi:10.1140/epjb/s10051-023-00575-2. [pp. 2, 4]

[DCRRM08] Gabriele De Chiara, Matteo Rizzi, Davide Rossini, and Simone Montangero. Density matrix renormalization group for dummies. *Journal of Computational and Theoretical Nanoscience*, 5(7):1277–1288, July 2008. URL: http://dx.doi.org/10.1166/jctn.2008.2564, doi:10.1166/jctn.2008.2564. [pp. 1, 2, 9]

[GM17] James R. Garrison and Ryan V. Mishmash. simple-dmrg/simple-dmrg: Simple dmrg 1.0, November 2017. doi:10.5281/zenodo.1068359. [p. 5]

[HUA+24] Johannes Hauschild, Jakob Unfried, Sajant Anand, Bartholomew Andrews, Marcus Bintz, Umberto Borla, Stefan Divic, Markus Drescher, Jan Geiger, Martin Hefel, Kévin Hémery, Wilhelm Kadow, Jack Kemp, Nico Kirchner, Vincent S. Liu, Gunnar Möller, Daniel Parker, Michael Rader, Anton Romen, Samuel Scalet, Leon Schoonderwoerd, Maximilian Schulz, Tomohiro Soejima, Philipp Thoma, Yantao Wu, Philip Zechmann, Ludwig Zweng, Roger S. K. Mong, Michael P. Zaletel, and Frank Pollmann. Tensor network Python (TeNPy) version 1. *SciPost Phys. Codebases*, page 41, 2024. URL: https://scipost.org/10.21468/SciPostPhysCodeb.41, doi:10.21468/SciPostPhysCodeb.41. [p. 5]

[Pir21] Antonio Sergio Teixeira Pires. The heisenberg model. In *Theoretical Tools for Spin Models in Magnetic Systems*, 2053-2563, pages 1–1 to 1–16. IOP Publishing, 2021. doi:10.1088/978-0-7503-3879-0ch1. [pp. 2, 8]

[Sch11] Ulrich Schollwöck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics*, 326(1):96–192, January 2011. URL: http://dx.doi.org/10.1016/j.aop.2010.09.012, doi:10.1016/j.aop.2010.09.012. [pp. 4, 5]

[Vid08] G. Vidal. Class of quantum many-body states that can be efficiently simulated. *Physical Review Letters*, 101(11), September 2008. URL: http://dx.doi.org/10.1103/PhysRevLett.101.110501, doi:10.1103/physrevlett.101.110501. [p. 9]

[vV16] Solange Schrijnder van Velzen. Quantum fluctuations in antiferromagnetic spin configurations. 2016. URL: https://studenttheses.uu.nl/bitstream/handle/20.500.12932/21911/ScriptieFinal.pdf. [p. 8]

[Whi92] Steven R. White. Density matrix formulation for quantum renormalization groups. *Phys. Rev. Lett.*, 69:2863–2866, Nov 1992. URL: https://link.aps.org/doi/10.1103/PhysRevLett.69.2863, doi:10.1103/PhysRevLett.69.2863. [pp. 1, 2, 3, 4]

[Wil75] Kenneth G. Wilson. The renormalization group: Critical phenomena and the kondo problem. *Rev. Mod. Phys.*, 47:773–840, Oct 1975. URL: https://link.aps.org/doi/10.1103/RevModPhys.47.773, doi:10.1103/RevModPhys.47.773. [p. 2]